

A cycle-family disproof of Conjecture 00000000464

ColdSmile

October 4, 2026

Abstract

For the connected simple 2-regular graphs $G_k = C_{2^{k+2}}$, the number of spanning trees is 2^{k+2} , so its largest prime factor is always 2. Their orders tend to infinity. Consequently, every positive logarithmic lower bound fails eventually along this family. The accompanying Lean project proves the spanning-tree formula from actual graph definitions and refutes even an eventual version of the proposed bound.

1 Statement, conventions, and scope

The English conjecture states:

Definition: $P(\tau(G))$ is the largest prime factor of $\tau(G)$. Conjecture: For families of connected d -regular graphs (vertex count $v \rightarrow \infty$), $P(\tau(G)) \geq c(d) \cdot \log v$ for an explicit $c(d) > 0$; and on random d -regular graphs, the distribution of $P(\tau(G))/\log v$ converges to an exponential distribution.

The original English and Chinese texts are preserved in `conjecture.md`. Neither excludes $d = 2$. We use $\tau(G)$ in its standard graph-theoretic sense: the number of spanning trees of G . A spanning tree is a connected acyclic subgraph retaining every vertex; distinct edge sets count separately. The formalization uses exactly this meaning. The logarithm is natural; changing its base to any fixed number greater than one only rescales a positive constant.

We refute the deterministic lower-bound clause. This suffices to disprove the stated conjunction; no claim about the random-graph distribution clause is needed. The counterexample relies openly on the permitted degree 2 and does not address a modified statement restricted to $d \geq 3$.

2 An admissible family

For $k \in \mathbb{N}$, put $n_k = 2^{k+2}$ and $G_k = C_{n_k}$. The cycle C_n has vertices $\mathbb{Z}/n\mathbb{Z}$ and an undirected edge between consecutive residues. Since $n_k \geq 4$, every vertex has exactly two distinct neighbors, and the cycle is connected. Thus (G_k) is a family of connected simple 2-regular graphs. Moreover $n_k \rightarrow \infty$.

In Lean these are Mathlib's `SimpleGraph.cycleGraph` on `Fin n`. Its adjacency is difference 1 in either orientation, and the formal proof establishes both connectedness and degree exactly 2.

3 Counting the actual spanning trees

Lemma 1. *For every integer $n \geq 3$, $\tau(C_n) = n$.*

Proof. The degree-sum formula gives $|E(C_n)| = n$. For any edge e , the graph $C_n - e$ is connected: after a cyclic relabeling, it contains the path visiting the vertices in the order $0, 1, \dots, n-1$. More precisely, if the removed edge is $\{d-1, d\}$, the translation $x \mapsto x + d$ maps the ordinary path on n vertices onto a spanning connected subgraph of $C_n - e$. No path edge maps to the removed wraparound edge. Every cycle edge has this form.

The graph $C_n - e$ has $n-1$ edges. A finite connected graph on n vertices with $n-1$ edges is a tree, so each deletion gives a spanning tree.

Conversely, a spanning tree $T \leq C_n$ has $n-1$ edges. Because its edge set is a subset of the n cycle edges, its complement consists of exactly one edge e , and $T = C_n - e$. Distinct deleted edges give distinct edge sets. Hence $e \mapsto C_n - e$ is a bijection from the n cycle edges to all its spanning trees. $\square$

The Lean definition of the count is

$$\text{Nat.card}\{T : \text{SimpleGraph } V \mid T \leq G \wedge T.\text{IsTree}\}.$$

Here V is the full vertex type of G . The proof constructs the above bijection; it does not define the count to equal n , count trees up to isomorphism, or assume a matrix-tree formula.

4 Failure of every positive eventual bound

Theorem 1. *There is no real $c > 0$ and natural number K such that*

$$c \log |V(G_k)| \leq P(\tau(G_k)) \quad \text{for every } k \geq K.$$

Proof. The lemma gives $\tau(G_k) = 2^{k+2}$. Because 2 is prime and $k+2 > 0$, the set of prime divisors of this number is exactly $\{2\}$. Its largest prime factor is therefore 2.

For any fixed $c > 0$, the expression $c \log n_k$ tends to positive infinity. In particular, there exists M such that $c \log n_k > 2$ for every $k \geq M$. If an eventual lower bound held from K onward, applying both inequalities at $k = \max(K, M)$ would give $2 < c \log n_k \leq 2$, a contradiction. $\square$

This refutes even a constant allowed to depend on this particular family and even a bound required only for sufficiently large members. It therefore refutes the proposed positive $c(2)$ depending solely on the degree. No finite numerical experiment is used in this argument.

5 Formal verification and correspondence

The project pins Lean 4.19.0 and Mathlib v4.19.0 at commit

`c44e0c8ee63ca166450922a373c7409c5d26b00b`

The complete proof is split into four modules:

- `CycleFacts.lean`: deleting any edge of an arbitrary cycle of order at least three leaves a connected graph, proved using a translated spanning path.
- `CycleTrees.lean`: a bijection from edges to spanning trees and the theorem `cycle_spanningTreeCount_of_three_le`.
- `NumberTheory.lean`: the orders 2^{k+2} tend to infinity, their largest prime factor is 2, and every positive logarithmic bound eventually exceeds 2.
- `Conjecture464.lean`: the actual graph family and the concluding theorem `conjecture464_counterexample`.

The final theorem bundles the following three facts about the same explicit family:

1. every graph is connected and 2-regular;
2. its vertex counts tend to infinity;
3. no positive constant and starting index make the logarithmic lower bound hold thereafter.

The function `largestPrimeFactor` is the supremum of the finite set `Nat.primeFactors`. Its convention on an empty set is irrelevant: every count in the family is a positive power of two, at least four. The noncomputable `Nat.card` tree count is evaluated by a proved finite bijection, rather than by unverified computation.

6 Reproduction

From the submitted `lean/` directory, install the pinned Lean toolchain and fetch dependencies, then run:

```
lake exe cache get
lake build
for source in CycleFacts.lean CycleTrees.lean \
               NumberTheory.lean Conjecture464.lean Check.lean
do
  lake env lean -DwarningAsError=true "$source" || exit 1
done
```

The committed dependency manifest fixes the dependency revisions. If the native cache executable cannot load on a platform, the README supplies the equivalent source-run cache command. `Check.lean` prints the main definitions, the full final theorem type, and the axiom dependencies of eleven central results.

All proof steps are checked by Lean’s kernel. The audit permits only the standard foundational axioms `propext`, `Classical.choice`, and `Quot.sound`; it uses no admitted proofs, custom axioms, or native-evaluation shortcut. No auxiliary numerical computation is required. The accompanying verification record describes the clean rebuild, source replay, and document checks. Those are local validation results, not a statement of maintainer acceptance.